SmartSplit — DSA & OOP Viva Prep

Nov 3, 2026 · @Muhammad Azeem

How to use this guide

Examiners ask two kinds of DSA and OOP questions: "What is X?" and "Where did you use X in your project?" This guide answers both, in simple words, with a SmartSplit example for each idea.

- **Short on time?** Read sections 4, 5, 8 and 9. They are about your own project, and that is what examiners care about most.
- **Have 1 hour?** Read everything from top to bottom, then do the self-test in section 10.
- **Golden rule:** first give a one-line meaning, then a SmartSplit example. For example: "A Map stores key and value pairs. In SmartSplit, `computeNetBalances` uses a Map from user ID to balance."
- If you don't know an answer, say: "I haven't used that in this project, but the idea is …". Never make something up.

1. Data structures in simple words

A **data structure** is a way of keeping data in memory so that the jobs you do most often are fast. The best example is a cupboard: you can throw clothes in a pile, or fold them into drawers. Drawers take more effort to fill, but finding a shirt is much faster.

Structure	What it is	Everyday example	In SmartSplit
Array	Items in a row. Each has a number (index) starting at 0.	A row of seats in a bus: seat 0, seat 1, seat 2	The `splits` list in an expense, and the line items of a receipt
Hash map (JS `Map` or object)	Key goes in, value comes out, almost instantly	A dictionary: look up a word, get its meaning	`computeNetBalances` keeps user ID → net balance in a `Map`
Set	A list where nothing can repeat	A class register: each name is written once	`new Set()` removes duplicate invite emails and duplicate categories

Structure	What it is	Everyday example	In SmartSplit
Stack	Last in, first out (LIFO)	A pile of plates: you take the top one first	The browser's Back button history, and the JavaScript call stack
Queue	First in, first out (FIFO)	A line at a bank counter	The Node.js event loop runs waiting tasks in order
Linked list	Each item points to the next one	A treasure hunt: each clue tells you where the next clue is	Not used directly. JS arrays already do this job for us
Tree	One root, branches below it, no loops	A family tree, or folders on a computer	MongoDB indexes are B-trees; React components form a tree
Graph	Points (nodes) joined by lines (edges)	A city map: places joined by roads	Group debts: each person is a node, each "A owes B" is an edge
Heap	A tree that always keeps the biggest (or smallest) item on top	A hospital emergency line: the most serious patient goes first	Not used. Sorting each step is fast enough for group sizes (comment in `settlement.js`)

If asked "Which data structure did you use most?" Say: hash maps (JavaScript `Map`) and arrays. A Map gives O(1) lookup, so adding up balances for 100 expenses stays fast.

2. Big-O: how fast does it grow?

Big-O tells us how the work grows when the input grows. It is not seconds. It is the shape of the growth. *n* means "how many items".

Story: you need to find your friend's name in a class list.

Big-O	Name	Class-list story	SmartSplit example
O(1)	Constant	You know they sit in seat 5. You go straight there.	`net.get(userId)` on a Map
O(log n)	Logarithmic	The list is sorted A–Z. Open the middle, throw away half, repeat.	MongoDB finds an expense by an index (B-tree)

Big-O	Name	Class-list story	SmartSplit example
O(n)	Linear	Read every name one by one.	`computeNetBalances` reads each split once
O(n log n)	Linearithmic	Sort the whole list first.	`debtors.sort(byLargest)`
O(n²)	Quadratic	Compare every student with every other student.	Without optimisation, everyone pays everyone: up to n(n−1)/2 pairs

Complexity of our own code (V = members in a group, S = all split rows in the group):

- `computeNetBalances`: **O(S)**. One pass over every split and payer.
- `minCashFlow`: the loop runs at most V−1 times, and each time it sorts both lists (V log V). Total **O(V² log V)**. For a trip of 10 people this is tiny.
- With a heap it would be **O(V log V)**. We chose sorting because it is simpler and groups are small. This trade-off is written in the code comment.

Space complexity means extra memory used. `minCashFlow` keeps two lists of size at most V, so its space is **O(V)**.

3. Sorting, searching, greedy, recursion

Sorting: putting things in order

Algorithm	How it works, simply	Speed
Bubble sort	Compare two neighbours and swap if wrong. Repeat until no swaps.	O(n²), slow
Selection sort	Find the smallest, put it first. Find the next smallest, and so on.	O(n²), slow
Merge sort	Cut the list in half again and again, then merge the sorted halves.	O(n log n), always
Quick sort	Pick a "pivot". Smaller items go left, bigger go right. Repeat on each side.	O(n log n) on average, O(n²) worst case

What does SmartSplit use? JavaScript's built-in `array.sort()`. In Chrome and Node.js (the V8 engine) it is **TimSort**, a mix of merge sort and insertion sort. It is O(n log n) and *stable* (equal items keep their order). We sort debtors by amount, alternatives by price, and lists by date.

Searching: finding one thing

- **Linear search, O(n):** check items one by one. Example: `group.memberFor(userId)` walks the members list to find the active member.
- **Binary search, O(log n):** only works on a sorted list. Look in the middle, keep the half that can hold the answer. Like finding a word in a dictionary. MongoDB indexes give us this speed for free.

Greedy algorithm: take the best choice right now

A greedy algorithm makes the best-looking choice at each step and never goes back.

Example: a shopkeeper gives Rs 1,700 change using the biggest notes first: one Rs 1,000, one Rs 500, two Rs 100. That is greedy.

SmartSplit: Min-Cash-Flow is greedy. At each step it matches the person who owes the **most** with the person who is owed the **most**. Section 5 explains it in full.

Recursion: a function that calls itself

Recursion solves a big problem by solving a smaller copy of the same problem. It always needs a **base case** to stop, like Russian dolls: you open dolls until you reach the smallest one.

```
function factorial(n) {
  if (n <= 1) return 1;        // base case: stop here
  return n * factorial(n - 1); // smaller copy of the same problem
}
```

SmartSplit: we use loops, not recursion. A loop is easier to read here and cannot overflow the call stack.

Dynamic programming (if they ask)

Remember the answers to small problems so you never solve the same one twice. Example: Fibonacci with a memo table. We did not need it in SmartSplit.

4. Where DSA is used inside SmartSplit

The answer to "Where is DSA in your project?" is this table. Learn the first four rows well. They are the most likely questions.

Feature	DSA idea	File	In one line
Who owes how much	Hash map, one pass, O(S)	`server/src/services/settlement.js` → `computeNetBalances`	Each payment adds to a person's total, and each share takes away from it
Settle up	Greedy + sorting	`settlement.js` → `minCashFlow`	Biggest debtor pays biggest creditor, repeat. At most V−1 payments
"6 payments → 2" stat	Map with a joined key, debtor + creditor	`settlement.js` → `rawPairwiseDebts`	Counts the payments needed without optimisation
Equal split	Integer division + remainder	`server/src/services/splitCalculator.js` → `distribute`	Rs 1,000 ÷ 3 = Rs 333 each, and the extra Rs 1 goes to the payer
Receipt item matching	String similarity (Dice coefficient on 2-letter pieces)	`server/src/services/priceMatch.js` → `similarity`	Score from 0 to 1; 0.8 or more counts as the same product
Cheapest price	Filter + sort, take top 3	`priceMatch.js` → `cheapest`	Fresh prices first, then sorted by price

Feature	DSA idea	File	In one line
Store-price checking	Set of words, subset test	`server/src/services/priceSync.js` → `sameProduct`	Every word of the product name must be in the store's title
Remove duplicates	Set	`groups.controller.js`, `categories.js`	The same invite email is never sent twice
Fast database lookups	B-tree indexes	`server/src/models/*.js` (`schema.index`)	E.g. expenses by group + date, found without scanning everything
Old login tokens deleted	TTL index	`server/src/models/RefreshToken.js`	MongoDB deletes a token by itself when it expires
Many requests, one refresh	One shared promise (waiters line up)	`client/src/api/client.js` → `refreshing`	If 5 requests fail at once, only one refresh call is made

The similarity score, with numbers

Split each word into 2-letter pieces (bigrams):

- "milk" → mi, il, lk (3 pieces)
- "milkk" (a typo on a receipt) → mi, il, lk, kk (4 pieces)
- Shared pieces: 3

Score = (2 × shared) ÷ (pieces in A + pieces in B) = (2 × 3) ÷ (3 + 4) = 6 ÷ 7 = 0.86

0.86 is at least 0.8, so "milkk" matches "milk". The pieces are counted in a `Map`, so this is fast.

Balances are computed, never stored

We never keep a running "Ali owes Rs 500" number in the database. Every time, balances are worked out again from the expenses, using a MongoDB **aggregation pipeline** (`server/src/services/balances.js`). It works like a factory line: filter the rows, then group them by person, then add them up. Why: a stored total can drift out of date. A computed one is always right.

5. Min-Cash-Flow, step by step

Min-Cash-Flow turns many small debts into the fewest payments it can, and never more than V−1 (V = people in the group). It is a greedy algorithm that uses sorting.

The story: a Murree trip with 4 friends

Expense	Paid by	Amount	Each person's share
Hotel	Azeem	Rs 6,000	Rs 1,500
Food	Chanda	Rs 2,400	Rs 600
Petrol	Saba	Rs 1,600	Rs 400

Everyone's share of the whole trip is 1,500 + 600 + 400 = **Rs 2,500**.

Step A: net balance = what you paid − your share

Person	Paid	Share	Net	Meaning
Azeem	Rs 6,000	Rs 2,500	**+3,500**	gets money back
Chanda	Rs 2,400	Rs 2,500	**−100**	owes
Saba	Rs 1,600	Rs 2,500	**−900**	owes
Bilal	Rs 0	Rs 2,500	**−2,500**	owes

Check: +3,500 − 100 − 900 − 2,500 = **0**. The total must always be zero. If it is not, the code throws an error instead of making wrong payments.

Step B: the greedy loop

1. Sort. The biggest creditor is Azeem (+3,500). The biggest debtor is Bilal (2,500).

2. Bilal pays Azeem **Rs 2,500** (the smaller of the two). Bilal is now 0 and leaves. Azeem still needs 1,000.
3. Sort again. Saba (900) pays Azeem **Rs 900**. Saba leaves. Azeem still needs 100.
4. Chanda pays Azeem **Rs 100**. Both are now 0. Done.

Result: 3 payments. Without the algorithm, each person would pay back each payer separately: **9 payments**. The app shows this as "9 → 3".

Why never more than V−1 payments?

- Every payment makes **at least one** person's balance zero, and that person leaves the list.
- The last payment makes **two** people zero together, because all balances add up to zero.
- So 4 people need at most 3 payments, and 10 people at most 9.

The code idea

```
while (creditors.length && debtors.length) {
  creditors.sort(byLargest);
  debtors.sort(byLargest);
  const amount = Math.min(creditors[0].net, debtors[0].net);
  payments.push({ from: debtors[0].userId, to: creditors[0].userId, amount
});
  creditors[0].net -= amount;
  debtors[0].net -= amount;
  if (creditors[0].net === 0) creditors.shift(); // square, leaves the list
  if (debtors[0].net === 0) debtors.shift();
}
```

Hard follow-up questions

- **"Is it always the absolute minimum?"** Not always. Finding the true minimum is an NP-hard problem (it is like the subset-sum problem). Greedy is fast and always stays within V−1, which is good enough for a trip.
- **"Why a graph?"** People are nodes and debts are edges. The algorithm replaces a messy graph with a simple one that has fewer edges.
- **"Why integers?"** Money is stored in whole rupees. Floats like 0.1 + 0.2 give 0.30000000000000004, and a few paisa could get lost.
- **"How do you know it works?"** Automated tests in `server/src/tests` check that there are at most V−1 payments and that everyone ends at exactly zero.

6. OOP basics in simple words

Object-Oriented Programming (OOP) means building software from "objects" that keep their own data and their own actions together.

Class and object

- **Class** = a blueprint, like a **cookie cutter**. It says what shape every cookie will have.
- **Object** = one real thing made from the class, like **one cookie**. One cutter can make many cookies.
- **Constructor** = the function that runs when a new object is made. It fills in the starting data.
- **`this`** = "me, this object". Inside a cookie, `this.flavour` means *my* flavour.

```
class Wallet {
  #balance = 0;                    // private: nobody outside can touch it
  constructor(owner) { this.owner = owner; }
  deposit(rupees) {
    if (!Number.isInteger(rupees) || rupees <= 0) throw new Error('Whole
rupees only');
    this.#balance += rupees;
  }
  get balance() { return this.#balance; }
}

const azeem = new Wallet('Azeem');   // object made from the class
azeem.deposit(500);
azeem.balance;                        // 500
```

The 4 pillars

Pillar	Simple meaning	Everyday example	In the Wallet code
Encapsulation	Keep the data and the code that changes it in one box. Hide the inside.	A medicine capsule: you swallow it, you don't touch the powder	`#balance` is private; you can only change it through `deposit`
Abstraction	Show only what is needed, hide the hard details	A car: you use the steering wheel and pedals, not the engine parts	You call `deposit(500)` and don't need to know how it is checked

Pillar	Simple meaning	Everyday example	In the Wallet code
Inheritance	A child class gets everything from a parent class and can add more	A child inherits a family name from their parents	`class SavingsWallet extends Wallet { … }`
Polymorphism	One name, many forms. The same call behaves differently for different objects	"Open" a door, a book, a bottle: the same word, different actions	Each class can have its own `toString()`, and all of them are called the same way

Other words examiners like

- **Method overriding:** a child class writes its own version of a parent's method. Same name, same inputs. This is decided at run time.
- **Method overloading:** the same method name with different inputs, e.g. `add(a, b)` and `add(a, b, c)`. Java and C++ support it. **JavaScript does not**: the last definition wins, so we use default values or check the inputs instead.
- **Access modifiers:** `public` (anyone), `private` (only this class), `protected` (this class and its children). In JavaScript, fields are public by default and `#name` makes them private. JS has no `protected`.
- **Abstract class:** a class you cannot make objects from; it is only a base for children. **Interface:** a list of methods a class promises to have. Java has both. JavaScript has neither as keywords. TypeScript adds `interface`.
- **Static method:** belongs to the class itself, not to one object. You call `ClassName.method()`.
- **Composition vs inheritance:** "has a" vs "is a". A car **has an** engine (composition). A car **is a** vehicle (inheritance). Prefer composition when in doubt.

7. OOP in JavaScript

JavaScript supports OOP, but it is a multi-paradigm language: it also supports functional and procedural styles. SmartSplit uses classes where they fit and plain functions everywhere else.

"Is JavaScript a real OOP language?"

Say: "Yes. Almost everything in JavaScript is an object, even arrays and functions. But its OOP is **prototype-based**, not class-based like Java. The `class` keyword came in ES6 (2015). It is a cleaner way to write prototypes; underneath, it still works with prototypes."

Prototype, simply

Every object has a hidden link to a "parent" object called its **prototype**. If JavaScript can't find a property on the object, it looks at the prototype, then that one's prototype, and so on. This is the **prototype chain**.

Example: `[3, 1, 2].sort()` works because every array is linked to `Array.prototype`, and `sort` lives there.

Functions are objects too

In JavaScript a function can be stored in a variable, passed to another function, or returned from one. A function that takes or returns a function is a **higher-order function**.

SmartSplit example: `validate(schema)` in `server/src/middleware/validate.js` *returns* a new function (Express middleware) that checks the request body with Zod.

Hiding data without classes

- **ES modules:** only what a file `export`s can be used by other files. Everything else stays private to that file. This is encapsulation at the file level.
- **Closures:** a function remembers the variables around it. The `refreshing` variable in `client/src/api/client.js` is hidden inside that module and shared by every request.

Why not make everything a class?

- Express middleware and React hooks are designed around functions.
- Pure functions such as `minCashFlow(balances)` are easier to test: same input, same output, no hidden state.
- So: **classes for things with identity and behaviour** (errors, database models), **functions for calculations**.

8. Where OOP is used inside SmartSplit

All four pillars appear in real SmartSplit code. Open these files during the viva if you are asked to show them.

Pillar	Where	What it shows
Inheritance	`server/src/middleware/ApiError.js` : `class ApiError extends Error`	Our error gets everything a normal JS error has, and adds `status` and `code`
Inheritance	`server/src/services/splitCalculator.js` : `class SplitError extends Error`	A special error for wrong split amounts
Inheritance + overriding	`client/src/components/ErrorBoundary.jsx` : `extends Component`	Overrides React's `componentDidCatch` and `render`, so one broken page doesn't blank the whole app
Static methods	`ApiError.notFound()`, `ApiError.forbidden()`	Called on the class itself, they build ready-made errors
Polymorphism	`server/src/middleware/errorHandler.js`	One handler takes *any* error (ApiError, SplitError, a plain Error) and reads `err.status` the same way
Encapsulation	`server/src/models/User.js` : `toPublic()` and `toJSON`	The user object never sends `passwordHash` out. It decides what the outside sees
Encapsulation	`server/src/models/GroupExpense.js` : `pre('validate')`	The expense checks its own rule (shares must add up to the total) before it saves
Abstraction	`server/src/models/Group.js` : `memberFor(userId)`	Callers ask "is this person an active member?" without knowing how members are stored
Abstraction	`client/src/api/*.js`	Pages call `listGroups()`, never axios directly. The HTTP details are hidden

The main code to show

```
export class ApiError extends Error {          // inheritance
  constructor(status, code, message) {          // constructor
    super(message);                             // runs the parent (Error)
constructor
    this.status = status;
    this.code = code;
  }
  static notFound(message = 'Not found') {      // static method
    return new ApiError(404, 'NOT_FOUND', message);
  }
}
```

Mongoose models are classes

- A **schema** is the blueprint: which fields, which types, which rules.
- `mongoose.model('Group', groupSchema)` turns it into a **class** (a Model).
- `new GroupExpense({ … })` makes an **object** (a document). `expense.save()` is a method it inherits from Mongoose.
- `groupSchema.methods.memberFor = …` adds our own **instance method** to every group object.

React components

- Components are reusable building blocks, like LEGO pieces. `StatCard` is written once and used on many pages.
- **Props** are the inputs, a bit like constructor arguments.
- Pages are made by **composition**: `AppLayout` contains the menu, and the page goes inside it. This is "has a", not "is a".
- `AuthContext` **encapsulates** login state. Any page can ask "who is logged in?" without knowing about tokens.

Layers: each one hides the details of the next

1. **Route** (`server/src/routes`): which URL, and Zod checks the input.
2. **Controller** (`server/src/controllers`): handles the request and sends the response.
3. **Service** (`server/src/services`): the real logic, such as `minCashFlow` .
4. **Model** (`server/src/models`): talks to MongoDB.

This separation of concerns is close to **MVC**: the Model is Mongoose, the View is React, and the Controller is Express.

9. Likely viva questions with short answers

DSA

1. **What is a data structure?** A way to store data so the jobs you do often are fast. Example: a Map for user ID → balance.
2. **Array vs linked list?** An array keeps items side by side, so reading item 5 is instant (O(1)). A linked list's items point to each other, so adding in the middle is cheap but reading item 5 means walking from the start (O(n)).
3. **Stack vs queue?** Stack: last in, first out, like plates. Queue: first in, first out, like a bank line.
4. **What is hashing?** A function turns a key into a slot number, so you can jump straight to the value. JavaScript's `Map` does this for us.
5. **What is Big-O?** How the work grows as the input grows. Our balance calculation is O(n), and sorting is O(n log n).
6. **Best sorting algorithm?** It depends. Merge sort is always O(n log n). Quick sort is usually fastest in practice. JavaScript's `sort()` uses TimSort.
7. **Binary search?** Only on sorted data. Check the middle and drop half each time: O(log n).
8. **What is a greedy algorithm? Where did you use it?** It takes the best choice at each step. Our Min-Cash-Flow settle-up is greedy.
9. **Time complexity of your settlement algorithm?** $O(V^2 \log V)$, because there are at most $V-1$ rounds and each sorts. A heap would make it $O(V \log V)$, but groups are small.
10. **Why at most V−1 payments?** Each payment clears at least one person, and the last one clears two.
11. **Is your algorithm optimal?** It always stays within $V-1$, but the true minimum is NP-hard to find, so greedy is the practical choice.
12. **Which data structure is the group-debt problem?** A graph. People are nodes and debts are edges.
13. **How does receipt item matching work?** It cleans the text, then compares 2-letter pieces with the Dice coefficient. A score of 0.8 or more is a match, and the pack sizes must also agree.
14. **How is the database fast?** Indexes. MongoDB stores them as B-trees, so a lookup is O(log n) instead of reading every row.

15. **Why integer rupees, not floats?** 0.1 + 0.2 is not exactly 0.3 in floating point. With whole rupees nothing gets lost, and any leftover rupee goes to the payer.

OOP

1. **What is OOP?** Building software from objects that keep their data and actions together.
2. **Class vs object?** A class is the cookie cutter; an object is the cookie.
3. **Name the 4 pillars.** Encapsulation, Abstraction, Inheritance, Polymorphism.
4. **Encapsulation vs abstraction?** Encapsulation *hides the data* inside a box. Abstraction *hides the complexity* behind a simple interface. Example: `toPublic()` hides the password hash (encapsulation); `memberFor()` hides how membership is checked (abstraction).
5. **Where is inheritance in your project?** `ApiError extends Error`, `SplitError extends Error`, `ErrorBoundary extends Component`.
6. **Where is polymorphism?** The error handler treats every kind of error the same way through `err.status` and `err.code`. ErrorBoundary overrides React's own methods.
7. **Overloading vs overriding?** Overloading: same name, different inputs (compile time; not in JavaScript). Overriding: a child replaces a parent's method (run time).
8. **What does `super()` do?** It calls the parent class's constructor. `ApiError` calls `super(message)` so that the Error part is set up.
9. **What is `this`?** The current object.
10. **What is a static method?** A method on the class, not on an object: `ApiError.notFound()`.
11. **Abstract class vs interface?** An abstract class can have some code and can't be made into an object. An interface only lists method names. JavaScript has neither keyword; TypeScript adds interfaces.
12. **Is JavaScript OOP?** Yes, prototype-based. `class` (ES6) is a neater way to write prototypes.
13. **Why are your React components functions, not classes?** Hooks make function components simpler. ErrorBoundary is the one exception, because React only offers error boundaries as classes.
14. **Composition vs inheritance in your project?** React pages are built by composition (a layout *has* a page inside it). Errors use inheritance (an ApiError *is an* Error).
15. **Which design pattern did you use?** MVC-style layers (route → controller → service → model), middleware (a chain of functions, also called chain of responsibility), and static factory methods in ApiError.

10. Self-test and last-minute cheat sheet

Self-test: say each answer out loud, then tick it

- [ ] Explain Min-Cash-Flow with the Murree example (4 people, 3 payments, "9 → 3").
- [ ] Why never more than V−1 payments?
- [ ] Name 3 data structures used in SmartSplit and say where.
- [ ] What is O(n log n), and where do we have it?
- [ ] Rs 1,000 split equally between 3 people: who gets the extra Rs 1, and why?
- [ ] How does "milkk" match "milk"?
- [ ] Name the 4 OOP pillars, each with a SmartSplit file.
- [ ] What does `super(message)` do in `ApiError`?
- [ ] Overloading vs overriding, and which one JavaScript has.
- [ ] Why are balances computed, not stored?

Cheat sheet

Word	One line	SmartSplit
Map	Key → value, O(1)	user ID → balance
Set	No repeats	unique invite emails
Sort	O(n log n), TimSort	biggest debtor first
Greedy	Best choice now	Min-Cash-Flow
Graph	Nodes + edges	people + debts
B-tree index	O(log n) lookup	`schema.index(...)`
Inheritance	Child gets the parent's things	`ApiError extends Error`
Encapsulation	Hide the data	`toPublic()` hides the password hash
Abstraction	Hide the complexity	`memberFor()`, `src/api` functions
Polymorphism	Same call, different objects	one error handler for every error
Static	Belongs to the class	`ApiError.notFound()`
Composition	"Has a"	`AppLayout` holds the page

Good luck. Speak slowly, give one example per answer, and open the code when they ask "where?".